UTEC
Universidad Tecnológica

Conocimientos adicionales

Búsqueda binaria

Concepto

La búsqueda binaria es un algoritmo eficiente para encontrar un elemento en una lista ordenada. Funciona dividiendo repetidamente el rango de búsqueda a la mitad, reduciendo así el tiempo de búsqueda a O(log n). Es aplicable solo a listas que están ordenadas previamente.

Pasos de la búsqueda binaria

1. Inicializar dos índices, low y high, que representan el rango de búsqueda.
2. Calcular el índice medio mid.
3. Comparar el elemento del medio con el elemento buscado.
4. Si el elemento buscado es igual al elemento del medio, devolver el índice.
5. Si el elemento buscado es menor que el elemento del medio, ajustar el índice high a mid -
6. Si el elemento buscado es mayor que el elemento del medio, ajustar el índice low a mid +
7. Repetir los pasos del 2 al 6 hasta que el elemento sea encontrado o el rango de búsqueda se reduzca a cero.

Ejemplo en Java

```java
public class BusquedaBinaria {
    public static int busquedaBinaria(int[] array, int clave) {
        int low = 0;
        int high = array.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2; // Evitar overflow
            if (array[mid] == clave) {
                return mid; // Elemento encontrado
            } else if (array[mid] < clave) {
                low = mid + 1;
            } else {
```

UTEC
Universidad Tecnológica

```
                high = mid - 1;
            }
        }
        return -1; // Elemento no encontrado
    }

    public static void main(String[] args) {
        int[] array = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
        int clave = 10;
        int resultado = busquedaBinaria(array, clave);
        if (resultado != -1) {
            System.out.println("Elemento encontrado en el índice: " +
resultado);
        } else {
            System.out.println("Elemento no encontrado.");
        }
    }
}
```

Concepto de Bit

Un bit (abreviatura de "binary digit", dígito binario en inglés) es la unidad básica de información en informática. Un bit puede tener solo dos valores posibles: 0 o 1. Estos valores representan los estados binarios básicos: apagado (0) y encendido (1). Los bits son la base de toda la información digital.

Representación de números en binario

El sistema binario es un sistema numérico en el que los números se representan utilizando solo dos dígitos: 0 y 1. Este sistema es fundamental en informática porque es el sistema nativo de los circuitos electrónicos de las computadoras.

Cada posición en un número binario representa una potencia de 2, comenzando con \(2^0\) en la posición más a la derecha. Aquí se muestra cómo se representa el número decimal 13 en binario:

UTEC
Universidad Tecnológica

- $2^3 = 8$
- $2^2 = 4$
- $2^1 = 2$
- $2^0 = 1$

Para convertir 13 a binario, buscamos las potencias de 2 que suman 13:

13 = 8 + 4 + 1

Representamos esto en binario como:

$$1101\ (1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0)$$

Entonces, 13 en decimal es igual a 1101 en binario.

Pasaje de números decimales a binarios

Para convertir un número decimal a binario, puedes seguir estos pasos:

1. División por 2: Divide el número por 2 y anota el resto.
2. Cociente: Usa el cociente obtenido como nuevo número a dividir.
3. Repetición: Repite el proceso hasta que el cociente sea 0.
4. Lectura: El número binario se obtiene leyendo los restos de abajo hacia arriba.

Ejemplo: Convertir 19 a Binario

1. 19 / 2 = 9 (resto 1)
2. 9 / 2 = 4 (resto 1)
3. 4 / 2 = 2 (resto 0)
4. 2 / 2 = 1 (resto 0)
5. 1 / 2 = 0 (resto 1)

Leyendo los restos de abajo hacia arriba, 19 en decimal es 10011 en binario.

UTEC
Universidad Tecnológica

Ejemplos de conversión

Decimal a Binario

- Decimal 7:
  - 7 / 2 = 3 (resto 1)
  - 3 / 2 = 1 (resto 1)
  - 1 / 2 = 0 (resto 1)
  - Binario: 111

- Decimal 25:
  - 25 / 2 = 12 (resto 1)
  - 12 / 2 = 6 (resto 0)
  - 6 / 2 = 3 (resto 0)
  - 3 / 2 = 1 (resto 1)
  - 1 / 2 = 0 (resto 1)
  - Binario: 11001

Binario a decimal

Para convertir un número binario a decimal, multiplica cada dígito por la potencia de 2 correspondiente y suma los resultados.

- Binario 1101:

- $1 * 2^3 = 8$
- $1 * 2^2 = 4$
- $0 * 2^1 = 0$
- $1 * 2^0 = 1$
- Decimal: $8 + 4 + 0 + 1 = 13$

- Binario 1010:

UTEC
Universidad Tecnológica

- $1 * 2^3 = 8$
- $0 * 2^2 = 0$
- $1 * 2^1 = 2$
- $0 * 2^0 = 0$
- Decimal: $8 + 0 + 2 + 0 = 10$

Resumen

- Un bit es la unidad más pequeña de datos en una computadora, representando un 0 o un 1.
- Los números binarios utilizan solo 0 y 1, con cada posición representando una potencia de 2.
- La conversión de decimal a binario implica dividir por 2 y recoger los restos.
- La conversión de binario a decimal implica sumar las potencias de 2 multiplicadas por los dígitos binarios correspondientes.

Comprender estos conceptos es esencial para trabajar con operaciones bit a bit y optimizar algoritmos a nivel de hardware en programación.

Operaciones Bit a Bit (Bitwise Operations)

Concepto

Las operaciones bit a bit (bitwise) se realizan directamente en los bits de los operandos. Estas operaciones son rápidas y se usan a menudo en programación de bajo nivel, criptografía, gráficos por computadora, y manipulación de bits. Las operaciones comunes incluyen AND, OR, XOR, NOT, desplazamiento a la izquierda y desplazamiento a la derecha.

Operadores Bit a Bit en Java

1. AND (&): Realiza una operación AND entre cada par de bits correspondientes de los operandos.

UTEC
Universidad Tecnológica

2. OR (|): Realiza una operación OR entre cada par de bits correspondientes de los operandos.
3. XOR (^): Realiza una operación XOR (exclusive OR) entre cada par de bits correspondientes de los operandos.
4. NOT (~): Realiza una operación NOT (complemento) sobre cada bit del operando.
5. Desplazamiento a la izquierda (<<): Desplaza los bits de un número a la izquierda.
6. Desplazamiento a la derecha (>>): Desplaza los bits de un número a la derecha (con signo).
7. Desplazamiento a la derecha sin signo (>>>): Desplaza los bits de un número a la derecha (sin signo).

Ejemplo en Java

```
public class OperacionesBitwise {
    public static void main(String[] args) {
        int a = 5;  // 0101 en binario
        int b = 3;  // 0011 en binario

        // AND bit a bit
        int and = a & b; // 0001 en binario (1 en decimal)
        System.out.println("a & b = " + and);

        // OR bit a bit
        int or = a | b; // 0111 en binario (7 en decimal)
        System.out.println("a | b = " + or);

        // XOR bit a bit
        int xor = a ^ b; // 0110 en binario (6 en decimal)
        System.out.println("a ^ b = " + xor);

        // NOT bit a bit
        int not = ~a; // 1010 en binario (en complemento a 2, -6 en
decimal)
        System.out.println("~a = " + not);

        // Desplazamiento a la izquierda
        int leftShift = a << 2; // 010100 en binario (20 en decimal)
        System.out.println("a << 2 = " + leftShift);

        // Desplazamiento a la derecha
        int rightShift = a >> 1; // 0010 en binario (2 en decimal)
```

UTEC
Universidad Tecnológica

```
        System.out.println("a >> 1 = " + rightShift);

        // Desplazamiento a la derecha sin signo
        int unsignedRightShift = a >>> 1; // 0010 en binario (2 en
decimal)
        System.out.println("a >>> 1 = " + unsignedRightShift);
    }
}
```

Detalles y ejemplos

1. AND (&):
   - Ejemplo: 5 & 3 produce 1.
   - Explicación: Compara bit a bit 0101 & 0011 = 0001.

2. OR (|):
   - Ejemplo: 5 | 3 produce 7.
   - Explicación: Compara bit a bit 0101 | 0011 = 0111.

3. XOR (^):
   - Ejemplo: 5 ^ 3 produce 6.
   - Explicación: Compara bit a bit 0101 ^ 0011 = 0110.

4. NOT (~):
   - Ejemplo: ~5 produce -6.
   - Explicación: Invierte los bits ~0101 = 1010 (en complemento a 2, esto representa -6).

5. Desplazamiento a la izquierda (<<):
   - Ejemplo: 5 << 2 produce 20.
   - Explicación: Desplaza los bits de 0101 dos posiciones a la izquierda = 010100.

6. Desplazamiento a la derecha (>>):
   - Ejemplo: 5 >> 1 produce 2.
   - Explicación: Desplaza los bits de 0101 una posición a la derecha = 0010.

7. Desplazamiento a la derecha sin signo (>>>):
   - Ejemplo: 5 >>> 1 produce 2.
   - Explicación: Desplaza los bits de 0101 una posición a la derecha sin considerar el signo = 0010.

UTEC
Universidad Tecnológica

Relación con las estructuras de datos

1. **Búsqueda Binaria**:
    - En **tablas hash**, aunque no se usa directamente, entender la búsqueda binaria ayuda a comprender las colisiones y la búsqueda eficiente.
    - En **arrays ordenados**, la búsqueda binaria es un método eficiente de búsqueda.
2. **Operaciones Bit a Bit**:
    - En **tablas hash**, las operaciones bit a bit son fundamentales para calcular funciones hash y manejar colisiones.
    - En **listas enlazadas**, **pilas** y **colas**, pueden usarse para optimizaciones y manipulación de datos a nivel de bits.